Geração automática de equações de estado a partir de Bond Graphs: protótipo e validação numérica

***Emelyn Clementino Freire*[1,2]**
***Arthur Phelipe de Oliveira Queiroz*[2]**
***Jaysa Keylla Siqueira Barbosa*[3]**
***Guilherme Souza de Farias*[1,2]**
***Sayonara Cristina de O. Magalhães*[2]**
***André Laurindo Maitelli*[2]**

[1] Programa de Pós-Graduação em Engenharia Elétrica e de Computação (PPgEEC), Centro de Tecnologia, Universidade Federal do Rio Grande do Norte (UFRN), Natal, RN, Brasil
[2] Departamento de Engenharia de Computação e Automação (DCA), UFRN, Natal, RN, Brasil
[3] Programa de Pós-Graduação em Sistemas e Computação (PPgSC), UFRN, Natal, RN, Brasil
<emelyn.freire.116@ufrn.edu.br>, <arthur.queiroz.115@ufrn.edu.br>, <jaysa.barbosa.108@ufrn.edu.br>, <guilhermefariias1995@gmail.com>, <sayonara.magalhaes.068@ufrn.edu.br>, <maitelli@dca.ufrn.br>

Resumo

A modelagem por Bond Graph representa sistemas dinâmicos de diferentes domínios físicos com um único conjunto de elementos, mas a passagem do diagrama às equações de estado costuma ser feita à mão ou em ferramentas comerciais. Este trabalho apresenta um protótipo em Python que recebe a descrição de um sistema (em JSON ou em texto simples em português), monta o Bond Graph, obtém simbolicamente as equações de estado, integra-as numericamente e verifica o resultado por autovalores e balanço de energia. As equações são obtidas resolvendo, de forma simbólica, o sistema algébrico formado pelas relações constitutivas dos elementos e pelas restrições das junções, o que também permite detectar causalidade derivativa. O protótipo foi avaliado em quatro sistemas lineares com solução analítica conhecida (circuitos RC e RLC, sistema massa-mola-amortecedor e motor de corrente contínua). O erro máximo relativo ficou entre $3{,}1\times10^{-12}$ e $4{,}3\times10^{-8}$; as ordens de convergência observadas para os métodos de Euler (1,00 a 1,08) e Runge-Kutta de quarta ordem (4,00 a 4,06) coincidiram com as teóricas; e seis integradores adaptativos do SciPy mantiveram o erro abaixo de 10^{-6}. Os resultados ainda não abrangem sistemas não lineares nem modelos de maior porte. Os experimentos são reproduzíveis em contêiner Docker com versões fixadas.

Abstract

Bond graph modelling represents dynamic systems from different physical domains with a single set of elements, but the step from the diagram to the state equations is usually done by hand or in commercial tools. This paper presents a Python prototype that takes a system description (JSON or plain Portuguese text), builds the bond graph, derives the state equations symbolically, integrates them numerically and checks the result through eigenvalues and an energy balance. The equations are obtained by symbolically solving the algebraic system formed by the element constitutive laws and the junction constraints, which also allows derivative causality to be detected. The prototype was evaluated on four linear systems with known analytical solutions (RC

and RLC circuits, a mass-spring-damper and a DC motor). The maximum relative error ranged from 3.1×10^{-12} to 4.3×10^{-8}; the observed convergence orders of the Euler (1.00 to 1.08) and fourth-order Runge-Kutta (4.00 to 4.06) methods matched the theoretical ones; and six adaptive SciPy integrators kept the error below 10^{-6}. The results do not yet cover nonlinear systems or larger models. The experiments can be reproduced in a Docker container with pinned versions.
Palavras-chave: Bond Graph; modelagem de sistemas dinâmicos; simulação numérica; equações de estado; Python.

1 Introdução

O Bond Graph, proposto por Paynter (1961), descreve um sistema físico pela troca de potência entre seus componentes. Cada ligação (*bond*) carrega um par de variáveis, esforço e fluxo, cujo produto é a potência, e um mesmo conjunto de elementos representa resistores, capacitores, massas, molas ou amortecedores. Por isso a técnica se aplica a sistemas que combinam domínios, como os eletromecânicos, sem trocar de formalismo (KARNOPP; MARGOLIS; ROSENBERG, 2012; BORUTZKY, 2010; GAWTHROP; BEVAN, 2007).

Depois de desenhado o diagrama, obter as equações de estado exige atribuir causalidade e combinar as relações de cada elemento, etapa sujeita a erros quando feita manualmente. Ferramentas como o 20-sim (BROENINK, 1999) automatizam esse processo em ambiente comercial. Entre as alternativas abertas, a biblioteca BondGraphTools, em Python, constrói modelos por composição simbólica com aplicações descritas sobretudo em biologia de sistemas (CUDMORE et al., 2019, 2021).

O protótipo descrito aqui não se propõe a substituir essas ferramentas. O 20-sim integra a entrada e o processamento de modelos hierárquicos, que combinam Bond Graphs e diagramas de blocos, verifica a consistência do modelo durante sua edição e dispõe de simulador próprio (BROENINK, 1999). O BondGraphTools é voltado à modelagem por script de sistemas multifísicos complexos e foi concebido para uso em conjunto com as demais bibliotecas científicas do Python (CUDMORE et al., 2019). O foco do protótipo está em três aspectos. Na entrada, o sistema pode ser descrito em alto nível, por uma árvore de ligações em série e em paralelo ou por um texto curto em português, e o Bond Graph é montado a partir dessa descrição. Na obtenção das equações, o sistema algébrico completo é resolvido simbolicamente, sem etapa explícita de atribuição de causalidade. Na verificação, um diagnóstico automático, por autovalores e balanço de energia, examina o resultado de cada simulação, e não apenas o modelo, e o conjunto é avaliado por um protocolo reproduzível contra soluções analíticas.

Este trabalho descreve um protótipo de simulador, também em Python, que cobre a cadeia completa: descrição do sistema, montagem do Bond Graph, geração simbólica das equações de estado, integração numérica e verificação automática do resultado. Nesta etapa, o objetivo é verificar, com métricas objetivas, se essa cadeia produz resultados corretos. As contribuições são: (i) a formulação da geração de equações como solução simbólica de um sistema algébrico linear, que dispensa a atribuição explícita de causalidade e sinaliza causalidade derivativa; (ii) um diagnóstico automático baseado em autovalores e em balanço de energia; e (iii) um protocolo experimental reproduzível, com comparação contra soluções analíticas e estudo de convergência.

2 Fundamentos

Em um Bond Graph, cada bond i tem esforço e_i e fluxo f_i, com potência $P_i = e_i f_i$. A meia seta indica o sentido em que a potência é considerada positiva. No domínio elétrico, esforço e fluxo correspondem a tensão e corrente; no mecânico translacional, a força e velocidade.

Os elementos de uma porta são as fontes de esforço (S_e) e de fluxo (S_f), o dissipador (R) e os armazenadores capacitivo (C) e inercial (I). Os armazenadores definem as variáveis de estado: o deslocamento generalizado q em cada C (carga, deslocamento) e o momento generalizado p em cada I (fluxo magnético, quantidade de movimento). Os elementos de duas portas, transformador (TF) e girador (GY), acoplam bonds sem armazenar nem dissipar energia. As junções impõem as leis de conservação: na junção 1, todos os bonds têm o mesmo fluxo e a soma algébrica dos esforços é nula (ligação em série); na junção 0, todos têm o mesmo esforço e a soma algébrica dos fluxos é nula (ligação em paralelo). A Tabela 1 resume as relações usadas pelo protótipo.

3 Protótipo

3.1 Arquitetura

O protótipo foi implementado em Python com NetworkX (HAGBERG; SCHULT; SWART, 2008), SymPy (MEURER et al., 2017) e SciPy (VIRTA-

Tabela 1: *Relações constitutivas implementadas.*

Elemento	Relação	Exemplos
S_e	$e = u(t)$	fonte de tensão; força
S_f	$f = u(t)$	fonte de corrente
R	$e = R\,f$	resistor; amortecedor
C	$e = q/C,\ \dot{q} = f$	capacitor; mola ($C = 1/k$)
I	$f = p/I,\ \dot{p} = e$	indutor; massa
TF	$e_1 = n\,e_2,\ f_2 = n\,f_1$	transformador
GY	$e_1 = r\,f_2,\ e_2 = r\,f_1$	motor CC
0	e comum, $\sum \pm f = 0$	ligação em paralelo
1	f comum, $\sum \pm e = 0$	ligação em série

NEN et al., 2020), organizado nos módulos da Figura 1. A entrada pode ser um JSON com elementos e bonds explícitos, um JSON em forma de árvore série/paralelo ou um texto curto em português, como "um resistor de 100 ohms em série com um capacitor de 1 µF conectado a uma fonte de tensão de 10 V". O texto é interpretado por regras (palavras-chave, valores com prefixos de unidade e as expressões "em série" e "em paralelo"), e o programa exibe o que entendeu para conferência. Essa interpretação cobre um vocabulário restrito (resistor, capacitor, indutor, diodo, fontes de tensão e de corrente e, no domínio mecânico, massa, mola, amortecedor e força aplicados a um mesmo corpo) e topologias formadas apenas por ligações em série e em paralelo; descrições fora desse padrão podem ser interpretadas de forma incorreta. A taxa de acerto da interpretação não foi avaliada neste trabalho, e todos os experimentos relatados a seguir usaram descrições em JSON. Na conversão de árvores, ligações em série viram junções 1 e ligações em paralelo, junções 0. O Bond Graph é armazenado como grafo direcionado, em que o sentido de cada aresta é o sentido positivo da potência.

3.2 Geração das equações de estado

Para um Bond Graph com N bonds há $2N$ incógnitas, os esforços e fluxos de todos os bonds. Cada extremidade de bond contribui com exatamente uma equação: um elemento de uma porta fornece sua relação constitutiva, uma junção com k bonds fornece $k-1$ igualdades e uma soma, e TF e GY fornecem duas relações. O sistema resultante tem, portanto, $2N$ equações. Nas junções, o sinal de cada termo das somas é positivo quando o bond entra na junção.

Com os estados $\mathbf{x}$ (os q e p) e as entradas $\mathbf{u}(t)$ tratados como conhecidos, as equações formam um sistema linear $\mathbf{A}\,\mathbf{z} = \mathbf{b}(\mathbf{x}, \mathbf{u})$ nas incógnitas $\mathbf{z}$, cujos coeficientes dependem dos parâmetros. O protótipo resolve esse sistema simbolicamente com o SymPy e substitui a solução em $\dot{q} = f$ e $\dot{p} = e$, obtendo $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta})$ com os parâmetros $\boldsymbol{\theta}$ ainda simbólicos. Assim, não é preciso atribuir causalidade explicitamente; em contrapartida, o custo cresce com o tamanho do sistema simbólico, o que não foi avaliado para modelos grandes. Quando o sistema não tem solução única, há estados dependentes, situação que no Bond Graph corresponde à causalidade derivativa (por exemplo, fonte de tensão ideal em paralelo com capacitor). Nesse caso a simulação é interrompida com uma mensagem que aponta as causas típicas.

O motor de corrente contínua usado nos experimentos é controlado pela armadura (Figura 2a): com a corrente de campo i_f constante, a tensão de armadura v_a (denotada V no modelo) alimenta o circuito formado por L_a e R_a, e o rotor, de momento de inércia J, está sujeito a atrito viscoso de coeficiente b. A força contraeletromotriz e o torque são proporcionais, respectivamente, à velocidade angular e à corrente de armadura, com a mesma constante K_m (MEDEIROS; MAITELLI, 2010). No Bond Graph da Figura 2b, essa relação é representada pelo girador GY, que acopla o domínio elétrico ao mecânico. As equações geradas pelo protótipo para esse modelo foram

$$\dot{p}_{L_a} = V - \frac{R_a}{L_a}\,p_{L_a} - \frac{K_m}{J}\,p_J, \tag{1}$$

$$\dot{p}_J = \frac{K_m}{L_a}\,p_{L_a} - \frac{b}{J}\,p_J, \tag{2}$$

em que p_{L_a}/L_a é a corrente de armadura e p_J/J a velocidade angular, as mesmas equações obtidas pelas leis de Kirchhoff e de Newton.

Resistores não lineares também podem ser declarados, como o diodo descrito pelo modelo de Shockley (1949). Nesse caso a parte linear do sistema é resolvida simbolicamente, e as poucas incógnitas restantes são calculadas numericamente a cada avaliação de $\mathbf{F}$. Essa funcionalidade não foi validada quantitativamente neste trabalho.

3.3 Integração e diagnóstico

As equações são convertidas em funções numéricas e integradas pelos métodos adaptativos do SciPy, entre eles o Runge-Kutta de Dormand-Prince (RK45) (DORMAND; PRINCE, 1980), os implícitos Radau e BDF (HAIRER; WANNER, 1996) e o LSODA, que alterna entre métodos rígidos e não rígidos (PETZOLD, 1983). Para estudo de convergência, foram implementados ainda os métodos de passo fixo de Euler explícito e Runge-Kutta clássico de quarta ordem (RK4) (HAIRER; NØRSETT; WANNER, 1993).

Após cada simulação, o protótipo calcula os autovalores da jacobiana (por diferenças finitas), dos

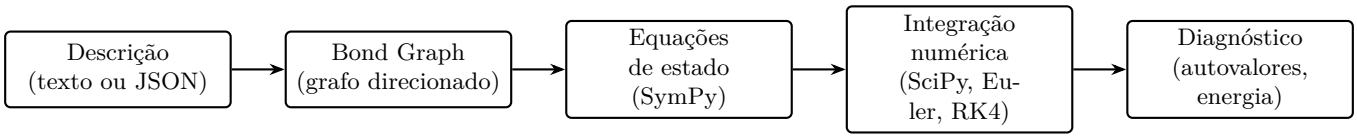


Figura 1: *Etapas do protótipo, da descrição do sistema ao diagnóstico do resultado.*

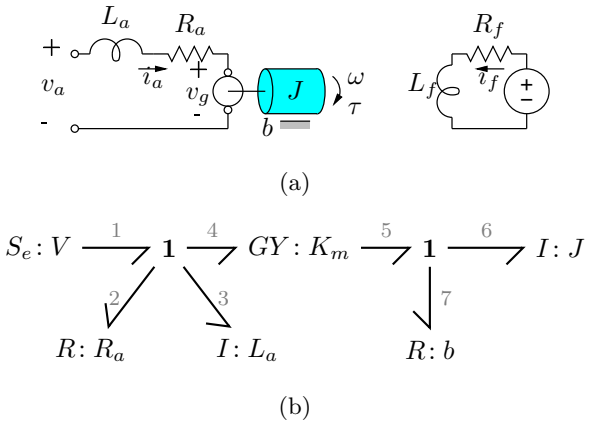


Figura 2: *Motor CC controlado pela armadura: (a) diagrama esquemático, adaptado de Medeiros e Maitelli (2010); (b) Bond Graph correspondente, com a parte elétrica à esquerda do girador e a mecânica à direita. Os números identificam os bonds.*

quais estima constantes de tempo, frequências naturais, fatores de amortecimento e a razão de rigidez, e emite avisos quando o tempo final ou o passo parecem inadequados. Calcula também o balanço de energia, comparando a energia fornecida pelas fontes, E_f, com a dissipada nos elementos R, E_d, e a variação da energia armazenada, $\Delta E_a = \Delta\big(\sum_C q^2/2C + \sum_I p^2/2I\big)$:

$$\varepsilon_E = \frac{|E_f - E_d - \Delta E_a|}{\max(|E_f|, |E_d|, |\Delta E_a|)}. \tag{3}$$

As energias E_f e E_d são integrais da potência no tempo, calculadas pela regra dos trapézios sobre os pontos de saída.

4 Metodologia experimental

Foram usados quatro sistemas lineares com solução analítica conhecida e condições iniciais nulas (Tabela 2). As soluções de referência foram escritas a partir dos parâmetros físicos, sem usar as equações geradas pelo protótipo: resposta ao degrau de primeira ordem para o RC, de segunda ordem subamortecida para o RLC e o massa-mola-amortecedor, e decomposição espectral da matriz do modelo do motor, montada a partir das leis de Kirchhoff e de Newton.

Tabela 2: *Sistemas avaliados, parâmetros, grandeza comparada e tempo final T.*

Sistema	Parâmetros	T
RC série (v_C)	$V = 10$ V, $R = 100\ \Omega$, $C = 1$ µF	0,5 ms
RLC série (v_C)	$V = 5$ V, $R = 10\ \Omega$, $L = 10$ mH, $C = 100$ µF	20 ms
Massa-mola-amortecedor (x)	$F = 10$ N, $m = 2$ kg, $k = 50$ N/m, $b = 3$ N·s/m	8 s
Motor CC (ω)	$V = 12$ V, $R_a = 1\ \Omega$, $L_a = 5$ mH, $K_m = 0{,}05$ N·m/A, $J = 2 \cdot 10^{-4}$ kg·m^2, $b = 10^{-4}$ N·m·s	0,4 s

Sendo y_k o valor simulado e y_k^* o analítico nos instantes de saída, adotaram-se o erro máximo relativo, $\varepsilon_{\max} = \max_k |y_k - y_k^*| / \max_k |y_k^*|$, e o NRMSE, raiz do erro quadrático médio dividida pela amplitude $\max_k y_k^* - \min_k y_k^*$. Três experimentos foram realizados:

E1 – precisão. Integração com RK45 (tolerâncias relativa 10^{-7} e absoluta 10^{-13}, passo máximo $T/200$) e 1000 pontos de saída; cálculo de $\varepsilon_{\max}$, NRMSE e ε_E.

E2 – convergência. Euler e RK4 com $N \in \{50, 100, \ldots, 3200\}$ passos ($h = T/N$). A ordem observada é a inclinação do ajuste por mínimos quadrados de $\log \varepsilon_{\max}$ contra $\log h$ nos quatro passos mais finos com $10^{-11} < \varepsilon_{\max} < 1$, o que exclui o piso de arredondamento e os casos instáveis.

E3 – métodos adaptativos. RK45, RK23, DOP853, Radau, BDF e LSODA com as mesmas tolerâncias; registro de $\varepsilon_{\max}$ e do número de avaliações de $\mathbf{F}$, incluindo as usadas na aproximação da jacobiana pelos métodos implícitos e no cálculo inicial dos autovalores.

Os resultados apresentados foram obtidos com Python 3.12, NumPy 2.5.3, SciPy 1.18.1 e SymPy 1.14.0. Os experimentos são executados por um único comando em contêiner Docker, sem acesso à rede, com uma CPU, versões fixadas e parâmetros lidos de um arquivo de configuração; cada execução registra o ambiente e uma assinatura SHA-256

Tabela 3: *E1: erro em relação à solução analítica e erro do balanço de energia (RK45).*

Sistema	$\varepsilon_{\max}$	NRMSE	ε_E
RC série	$3{,}13 \cdot 10^{-12}$	$5{,}82 \cdot 10^{-13}$	$2{,}12 \cdot 10^{-6}$
RLC série	$1{,}65 \cdot 10^{-9}$	$5{,}30 \cdot 10^{-10}$	$3{,}34 \cdot 10^{-5}$
Massa-mola-amort.	$4{,}27 \cdot 10^{-8}$	$1{,}72 \cdot 10^{-8}$	$1{,}34 \cdot 10^{-4}$
Motor CC	$1{,}48 \cdot 10^{-9}$	$2{,}10 \cdot 10^{-10}$	$3{,}00 \cdot 10^{-5}$

Tabela 4: *E2: ordem de convergência observada (teórica: 1 para Euler e 4 para RK4).*

Sistema	Euler	RK4
RC série	1,00	4,03
RLC série	1,01	4,01
Massa-mola-amortecedor	1,08	4,00
Motor CC	1,00	4,06

dos resultados numéricos, que permite verificar se duas execuções produziram os mesmos valores. O código-fonte será disponibilizado em repositório público após a avaliação do trabalho.

5 Resultados e discussão

5.1 Precisão

A Tabela 3 mostra que o erro máximo relativo ficou entre $3{,}1 \cdot 10^{-12}$ e $4{,}3 \cdot 10^{-8}$, valores compatíveis com a tolerância relativa de 10^{-7} do integrador. O maior erro ocorreu no massa-mola-amortecedor, possivelmente porque é o único caso em que a simulação cobre várias oscilações pouco amortecidas (fator de amortecimento 0,15). Como os quatro sistemas foram descritos com o mesmo conjunto de elementos, inclusive o motor, que combina dois domínios, o resultado indica que, nesses casos, as equações geradas automaticamente coincidem com as obtidas pelas leis físicas de cada domínio.

O erro do balanço de energia, entre $2 \cdot 10^{-6}$ e $1{,}3 \cdot 10^{-4}$, é várias ordens de grandeza maior que o erro dos estados. A diferença vem da quadratura: ao repetir as simulações com 10 000 pontos de saída em vez de 1000, ε_E caiu cerca de cem vezes nos casos RC, massa-mola-amortecedor e motor, como se espera da regra dos trapézios, cujo erro decresce com o quadrado do intervalo de amostragem. O balanço serve, assim, como verificação de consistência física, mas sua sensibilidade depende do número de pontos de saída.

5.2 Convergência

A Figura 3 e a Tabela 4 mostram que as ordens observadas coincidem com as teóricas: de 1,00 a 1,08 para Euler e de 4,00 a 4,06 para RK4. No massa-mola-amortecedor, Euler diverge nos dois maiores passos ($h = 0{,}16$ s e 0,08 s). Para os autovalores $\lambda = -0{,}75 \pm 4{,}94j$ desse sistema, $|1 + h\lambda|$ vale 1,18 e 1,02 nesses passos, fora da região de estabilidade do método, e 0,99 para $h = 0{,}04$ s. Esses pontos ficaram fora do ajuste da ordem. Para RK4 no circuito RC, o erro no passo mais fino ($2 \cdot 10^{-14}$) já se aproxima do piso de arredondamento.

Esse resultado verifica a implementação dos integradores de passo fixo e, de forma independente do E1, a coerência entre as equações geradas e as soluções analíticas: se houvesse erro nas equações, o erro numérico convergiria para um valor não nulo, e não para zero na taxa prevista.

5.3 Métodos adaptativos

Na Tabela 5, todos os métodos mantiveram $\varepsilon_{\max}$ abaixo de 10^{-6} (o maior valor, $6{,}3 \cdot 10^{-7}$, ocorreu com BDF no massa-mola-amortecedor). O DOP853 obteve os menores erros, de $5 \cdot 10^{-16}$ a $3 \cdot 10^{-11}$, com cerca de 3000 avaliações em todos os casos, e o LSODA usou o menor número de avaliações (366 a 1124). Nenhum dos quatro sistemas é rígido (a maior razão entre autovalores, no motor, é de cerca de 13). Nessas condições, o Radau exigiu mais avaliações que o RK45 em todos os casos, e o BDF usou menos avaliações, porém com erros maiores; o RK23, de ordem mais baixa, chegou a 8495 avaliações no massa-mola-amortecedor. O número de avaliações não depende da máquina e por isso foi preferido ao tempo de execução como medida de custo.

6 Limitações e trabalhos futuros

Os resultados se restringem a quatro sistemas lineares com solução analítica conhecida, com no máximo dois estados e sete bonds. Eles não validam, portanto, o tratamento de sistemas não lineares nem de modelos de maior complexidade. Ficam para trabalhos futuros: a comparação de casos não lineares, como o retificador com diodo, com ferramentas de referência; a avaliação do custo da solução simbólica em modelos com muitos bonds; a medição da taxa de acerto da entrada em texto sobre um conjunto de descrições; e o tratamento da causalidade derivativa, hoje apenas detectada. O protótipo inclui ainda módulos preliminares de ajuste de parâmetros a partir de dados medidos e de representação de elementos não lineares por redes neurais, testados apenas com dados sintéticos, que motivam uma linha de pesquisa

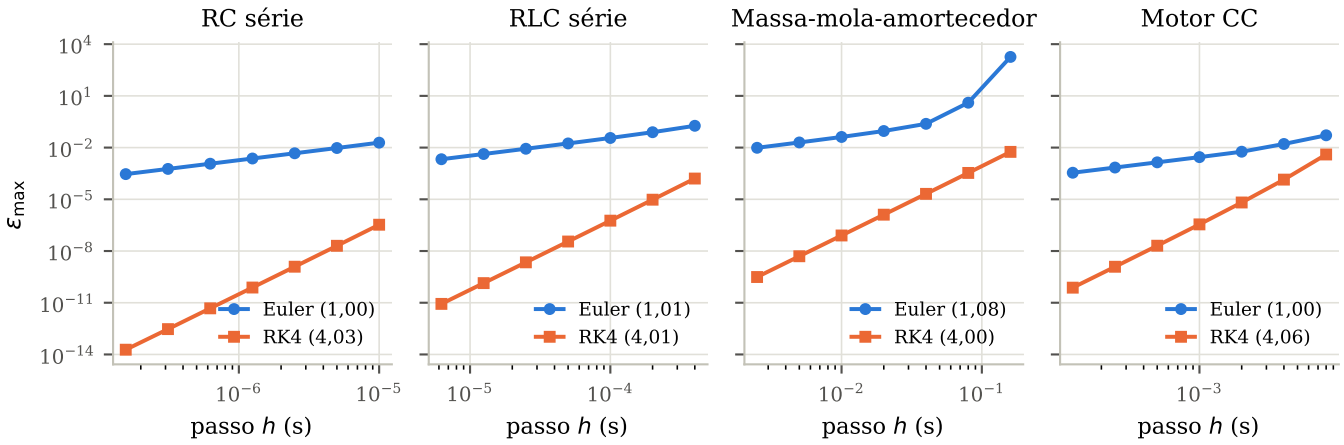


Figura 3: *E2: erro máximo relativo em função do passo para Euler e RK4. Entre parênteses, a ordem observada (inclinação ajustada nos quatro passos mais finos).*

Tabela 5: *E3: erro máximo relativo $\varepsilon_{\max}$ e número de avaliações de* **F** *(entre parênteses) para os métodos adaptativos do SciPy.*

Método	RC série	RLC série	Massa-mola-amortecedor	Motor CC
RK45	$3{,}13 \cdot 10^{-12}$ (1204)	$1{,}65 \cdot 10^{-9}$ (1235)	$4{,}27 \cdot 10^{-8}$ (1679)	$1{,}48 \cdot 10^{-9}$ (1307)
RK23	$9{,}11 \cdot 10^{-8}$ (730)	$7{,}52 \cdot 10^{-8}$ (3494)	$2{,}88 \cdot 10^{-7}$ (8495)	$4{,}65 \cdot 10^{-8}$ (1640)
DOP853	$5{,}37 \cdot 10^{-16}$ (3004)	$1{,}01 \cdot 10^{-14}$ (3020)	$8{,}91 \cdot 10^{-13}$ (3047)	$2{,}66 \cdot 10^{-11}$ (3035)
Radau	$4{,}11 \cdot 10^{-10}$ (1409)	$7{,}73 \cdot 10^{-9}$ (2380)	$1{,}16 \cdot 10^{-8}$ (5566)	$8{,}01 \cdot 10^{-10}$ (1902)
BDF	$3{,}85 \cdot 10^{-8}$ (445)	$1{,}40 \cdot 10^{-7}$ (686)	$6{,}30 \cdot 10^{-7}$ (1407)	$1{,}07 \cdot 10^{-8}$ (703)
LSODA	$1{,}35 \cdot 10^{-8}$ (404)	$5{,}74 \cdot 10^{-9}$ (477)	$1{,}11 \cdot 10^{-7}$ (1124)	$3{,}69 \cdot 10^{-8}$ (366)

sobre a integração de técnicas de aprendizado de máquina à modelagem por Bond Graph. Um editor gráfico e a avaliação do uso da ferramenta no ensino também estão previstos.

7 Conclusões

Este trabalho apresentou um protótipo que gera simbolicamente as equações de estado a partir de um Bond Graph e as integra numericamente. Nos quatro sistemas lineares avaliados, que incluem um modelo eletromecânico, o erro em relação às soluções analíticas ficou no nível da tolerância do integrador, as ordens de convergência de Euler e RK4 coincidiram com as teóricas e o balanço de energia fechou dentro do erro da quadratura. Esses resultados verificam a cadeia de geração e simulação apenas para sistemas lineares de pequeno porte. A validação com modelos não lineares e de maior complexidade é o passo seguinte, entre as extensões listadas na seção anterior.

Declaração de uso de inteligência artificial

Os autores utilizaram o assistente de inteligência artificial Claude (Anthropic) como apoio na implementação do protótipo, na preparação dos scripts dos experimentos e na redação deste texto. Todo o conteúdo gerado foi revisado e editado pelos autores, que assumem total responsabilidade pelo trabalho.

Referências

BORUTZKY, W. *Bond Graph Methodology: Development and Analysis of Multidisciplinary Dynamic System Models*. London: Springer, 2010.

BROENINK, J. F. 20-sim software for hierarchical bond-graph/block-diagram models. *Simulation Practice and Theory*, v. 7, n. 5–6, p. 481–492, 1999.

CUDMORE, P. et al. *Computer-aided modelling of complex physical systems with BondGraphTools*. 2019. ArXiv:1906.10799.

CUDMORE, P. et al. Analysing and simulating energy-based models in biology using BondGraphTools. *The European Physical Journal E*, v. 44, p. 148, 2021.

DORMAND, J. R.; PRINCE, P. J. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, v. 6, n. 1, p. 19–26, 1980.

GAWTHROP, P. J.; BEVAN, G. P. Bond-graph modeling: a tutorial introduction for control engineers. *IEEE Control Systems Magazine*, v. 27, n. 2, p. 24–45, 2007.

HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference (SciPy 2008)*. Pasadena, CA: [s.n.], 2008. p. 11–15.

HAIRER, E.; NØRSETT, S. P.; WANNER, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*. 2. ed. Berlin: Springer, 1993.

HAIRER, E.; WANNER, G. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*. 2. ed. Berlin: Springer, 1996.

KARNOPP, D. C.; MARGOLIS, D. L.; ROSENBERG, R. C. *System Dynamics: Modeling, Simulation, and Control of Mechatronic Systems*. 5. ed. Hoboken, NJ: John Wiley & Sons, 2012.

MEDEIROS, A. A. D.; MAITELLI, A. L. *Modelagem e Análise de Sistemas Dinâmicos: material didático*. Natal: Universidade Federal do Rio Grande do Norte, Departamento de Engenharia de Computação e Automação, 2010.

MEURER, A. et al. SymPy: symbolic computing in Python. *PeerJ Computer Science*, v. 3, p. e103, 2017.

PAYNTER, H. M. *Analysis and Design of Engineering Systems*. Cambridge, MA: MIT Press, 1961.

PETZOLD, L. Automatic selection of methods for solving stiff and nonstiff systems of ordinary differential equations. *SIAM Journal on Scientific and Statistical Computing*, v. 4, n. 1, p. 136–148, 1983.

SHOCKLEY, W. The theory of p-n junctions in semiconductors and p-n junction transistors. *Bell System Technical Journal*, v. 28, n. 3, p. 435–489, 1949.

VIRTANEN, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020.